The open-boundary Schwinger calculation
From CPU Julia to GPU DMRG

Implementation and technical guide for the supplied Julia programs

November 11, 2026
Repository packaging update; CPU validation: October 22, 2026

Abstract

This guide reconstructs the Hamiltonian implemented in the two supplied Julia files, explains how their matrix product operators and DMRG solver work, and documents a separate implementation with a portable CPU backend and an NVIDIA CUDA backend. The original files are not edited. Small CPU checks are distinguished from GPU validation and from the proposed $N = 10, \ldots, 20$ comparison. The computer inspected for this work has Intel UHD Graphics 620; it cannot execute the supplied CUDA backend. NVIDIA execution remains to be validated on a compatible device.

Contents

1 What has been implemented and what has not been measured **3**

2 The Hamiltonian in the original programs **3**

2.1 Local conventions and parameters 3

2.2 Electric charge and the gauge term 4

2.3 Why three virtual states represent a long-range interaction 4

3 MPS, MPO, and DMRG: what the numerical algorithm does **5**

3.1 Compression of the state 5

3.2 One two-site update 5

3.3 The numerical controls 5

3.4 Spin initialization and odd system sizes 6

4 Walkthrough of the two original files **6**

4.1 `run_OBC_Schwinger.jl` 6

4.2 `OBC_Schwinger_dmrg.jl`: construction utilities 6

4.3 The original diagnostics and solver 7

5 New files: what each one does **7**

5.1 Configuration and the general backend interface 8

5.2 Hamiltonian and starting state 8

5.3 The NVIDIA adapter 9

5.4 Optimization and observables 9

5.5 Running, parsing, and saving 10

5.6 Comparison runner 10

6 Correctness checks and their limits **10**

6.1 Independent tests before the study 10

6.2 Observed local validation 11

6.3 GPU checks that remain to be run 12

7 Commands for review, small checks, and the later study **12**

7.1 Portable CPU execution 12

7.2 NVIDIA setup and first real-device check 12

7.3 After reading the guide: $N = 10, \ldots, 20$ 12

7.4 Options and interpretation of outputs 13

8 Performance expectations and a defensible extended study **13**

8.1 Why small GPU runs can be slower 13

8.2 What is timed 13

8.3 Suggested research progression 14

9 Sources and reproducibility **14**

1 What has been implemented and what has not been measured

The new code is in `GPUimplementation/`. Unchanged reference copies of the two supplied CPU programs are in `reference/` inside that folder; the existing repository-root Julia programs are separate examples and are not modified. The original `run_OBC_Schwinger.jl` and `OBC_Schwinger_dmrg.jl` remain the reference implementation. The production algorithm is still matrix product state (MPS) DMRG. Moving this algorithm to GPU means putting the numerical tensor storage on the device and using the GPU implementations of contractions and decompositions. It does not mean constructing a dense many-body Hamiltonian on the GPU.

There are two concrete execution paths: the portable CPU program `run_cpu.jl`, and the NVIDIA program `run_cuda.jl`. Both call the same model builder, DMRG procedure, and diagnostics. A general `Backend` interface makes other device adapters possible, but no untested adapter is presented as a working Intel, AMD, or Apple implementation. The ITensor GPU documentation describes backend-dependent restrictions on matrix decompositions and precision [1]. In particular, an NVIDIA implementation is not portable to the Intel integrated GPU detected here.

The user requested implementation and explanation before the $N = 10$ through 20 study. Accordingly, only tiny correctness checks are part of this delivery. The study runner is ready as a separate command. There is no measured CUDA speedup or CPU/GPU agreement on this computer.

2 The Hamiltonian in the original programs

2.1 Local conventions and parameters

Each site is a spin-$\frac{1}{2}$ degree of freedom in the basis $(|\mathrm{Up}\rangle, |\mathrm{Dn}\rangle)$. ITensor's operators are

$$S^z = \frac{1}{2}\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \qquad S^+ = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \qquad S^- = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}. \tag{1}$$

Thus the original array `Z` is $2S^z$, not S^z. This factor matters when checking mass and interaction coefficients. The chain has N sites and $N-1$ dynamical links, with open boundaries and no coupling from site N to site 1.

Reading the code literally gives

$$H = yH_c + \mu H_m + xH_{xy}, \qquad \mu = 2\sqrt{x}\, m - \frac{1}{4}, \tag{2}$$

$$H_m = \sum_{j=1}^{N} (-1)^j S_j^z, \tag{3}$$

$$H_{xy} = \sum_{j=1}^{N-1} \left(S_j^+ S_{j+1}^- + S_j^- S_{j+1}^+ - 4g S_j^z S_{j+1}^z \right). \tag{4}$$

Here m denotes the input named `mass`. The g term is inside H_{xy}, so its coefficient in the complete Hamiltonian is $-4xg$. Moving it outside the factor x would change the model. Also, $S^+S^- + S^-S^+ = 2(S^xS^x + S^yS^y)$ on neighboring sites. No extra factor of two is introduced in the new code.

The default parameters are $x = 10$, $y = 1$, $m = 0.33$, $\theta = \pi$, $g = 0$. These formulas and the mass shift are preserved from the source. This guide does not independently establish their continuum renormalization convention or assign physical units to the resulting energies.

2.2 Electric charge and the gauge term

Define the staggered charge and background field by

$$q_j = S_j^z + \frac{(-1)^j}{2} I, \qquad \alpha = \frac{\theta}{2\pi}, \qquad L_n = \alpha + \sum_{j=1}^{n} q_j. \tag{5}$$

The custom MPO implements

$$H_c = \sum_{n=1}^{N-1} L_n^2. \tag{6}$$

The final site has no outgoing dynamical electric link. Consequently q_N does not appear in this electric energy, although the mass and hopping terms still act on site N. Constant terms from L_n^2, including the background-field contribution, are retained: absolute energy comparisons therefore have the same offset as the original.

For a product configuration $s_j = \pm\frac{1}{2}$, this operator is diagonal, with value

$$E_c(s_1, \ldots, s_N) = \sum_{n=1}^{N-1} \left[\alpha + \sum_{j=1}^{n} \left(s_j + \frac{(-1)^j}{2} \right) \right]^2 . \tag{7}$$

This provides an independent check of every diagonal matrix element. The hopping part swaps neighboring opposite spins with amplitude x; together these facts define the independent full matrix used only for tiny tests.

2.3 Why three virtual states represent a long-range interaction

Let $a_1 = q_1 + \alpha I$ and $a_j = q_j$ for $j > 1$. Since operators on distinct sites commute,

$$H_c = \sum_{i=1}^{N-1} (N-i) a_i^2 + 2 \sum_{1 \le i < j \le N-1} (N-j) a_i a_j. \tag{8}$$

To derive the coefficients, expand every $(a_1 + \cdots + a_n)^2$. A fixed a_i^2 occurs for $n = i, \ldots, N-1$, namely $N-i$ times. A pair $a_i a_j$ with $i < j$ occurs $N-j$ times, with factor two from the square.

The local operator-valued matrix is

$$W_j = \begin{pmatrix} I & a_j & (N-j)a_j^2 \\ 0 & I & 2(N-j)a_j \\ 0 & 0 & I \end{pmatrix} \quad (j < N), \qquad W_N = \mathrm{diag}(I, I, I). \tag{9}$$

The left boundary selects virtual state 1 and the right boundary selects state 3. In the product $W_1 \cdots W_N$, a path $1 \to 3$ at site i contributes $(N-i)a_i^2$. A path $1 \to 2$ at i, propagation in state 2, and $2 \to 3$ at j contributes $2(N-j)a_i a_j$. All other sites propagate identities. These are exactly the terms in Eq. (8). This is why a gauge interaction that is long range in the spin variables can have MPO bond dimension only three.

3 MPS, MPO, and DMRG: what the numerical algorithm does

3.1 Compression of the state

An arbitrary spin wavefunction contains 2^N amplitudes. An open-boundary MPS writes them as

$$\psi_{s_1 \cdots s_N} = \sum_{a_1,\ldots,a_{N-1}} A^{s_1}_{1a_1} A^{s_2}_{a_1 a_2} \cdots A^{s_N}_{a_{N-1}1}. \tag{10}$$

If each virtual dimension is at most χ, the bulk storage is of order $Nd\chi^2$, where $d = 2$. This is an approximation controlled by the entanglement across each cut, not simply by the number of sites. An exact state may require $\chi = 2^{\lfloor N/2 \rfloor}$ at the center. At $N = 20$, that upper bound is 1024; setting `maxdim=100` is therefore not an exact-diagonalization guarantee.

An MPO has two physical indices (input and output) per site and virtual dimension D. ITensor contracts matching index identities rather than relying on positional axis ordering. Priming the output physical index distinguishes it from the input index. The combined Hamiltonian is an exact direct sum of the three component MPOs; the virtual dimensions add and no compression error is introduced by this addition.

3.2 One two-site update

DMRG puts the MPS in a canonical form around two adjacent sites. Contracting all other sites into left and right environments defines an effective Hermitian Hamiltonian acting on the combined two-site tensor. A Krylov eigensolver approximates its lowest eigenvector without storing that effective matrix densely. The optimized tensor is split back into two MPS tensors, keeping at most `maxdim` Schmidt states and applying the truncation cutoff.

With noise enabled, ITensor can perturb the reduced density matrix used to choose the retained basis. Thus not every split is simply a noise-free SVD. Both the contraction and matrix-decomposition work are relevant to the GPU port. Sweeping left-to-right and right-to-left repeats these local optimizations. Since the optimization is variational within the current ansatz, a low energy is useful evidence, but a plateau alone does not establish the global ground state.

3.3 The numerical controls

The original defaults are 10 sweeps, maximum bond dimension 100, cutoff zero, noise 10^{-12}, Krylov dimension 10, and one allowed Krylov rebuild. The new defaults preserve those controls. A cutoff of zero does not remove errors from the bond-dimension cap or incomplete local solves. Raising N without testing larger χ and more sweeps can produce apparently smooth but unconverged results.

The observer stops when consecutive sweep energies obey

$$|E_k - E_{k-1}| \le \varepsilon \max(1, |E_k|), \quad k \ge k_{\min}, \tag{11}$$

with default $\varepsilon = 10^{-10}$ and $k_{\min} = 2$. This is safe near zero energy. Setting `energy_tol=0` disables early stopping, which is useful when comparing fixed sweep counts. The observer records each sweep's energy and maximum local truncation error. A fresh observer is created for each excited-state optimization.

3.4 Spin initialization and odd system sizes

The original active driver assumes even N. For N spin-$\frac{1}{2}$ sites, an initial total $S^z = M$ requires

$$n_{\mathrm{Up}} = N/2 + M \in \{0, 1, \ldots, N\}. \tag{12}$$

Thus $M = 0$ is impossible when N is odd. The new command-line default is $M = 0$ for even N and $M = +\frac{1}{2}$ for odd N. Positive-spin changes flip down spins from the right; negative-spin changes flip up spins from the left, reproducing the original even-chain conventions.

Crucially, the original uses `conserve_qns=false`. Its `spin` variable chooses an initial state, not a mathematically enforced optimization sector. Although the Hamiltonian conserves total S^z, dense numerics, basis expansion, and noise do not provide a sector constraint. The new dense GPU path preserves that behavior and records measured total S^z. For sector-resolved spectroscopy, a separate, validated QN implementation or an explicitly validated sector constraint is needed. Odd and even sizes also belong to different parity sequences; analyze that distinction in finite-size extrapolations.

4 Walkthrough of the two original files

4.1 `run_OBC_Schwinger.jl`

The imports load tensor types, MPS/MPO algorithms, linear algebra, and random-number utilities, then include the helper file. BLAS is set to four threads. The small matrices X, Y, Z, S^+, S^-, I define local spin conventions; only Z and I are needed by the active custom gauge builder. Threaded block-sparse contractions are enabled, but the active sites are dense, so that switch does not create a GPU path or QN conservation.

The parameter block sets the model and solver inputs, with active $N = 20$ and one requested state. The explicit branches construct a staggered product state with initial spin 0, ± 1, or ± 2 for even N. The odd-chain alternatives are commented out. `MPS(sites,state)` converts the labels into a bond-dimension-one MPS.

The first `OpSum` creates hopping and nearest-neighbor g interactions. The second creates the staggered mass operator. `fn_gauge` captures N and θ and passes each tensor to the custom gauge builder. `Schwinger` combines these components and runs DMRG. The following loops print norms, mutual overlaps, parameters, and energies. File writes and HDF5 checkpoint blocks are commented out, so they are not part of the active calculation.

Finally, the MPS is canonicalized at $\lfloor N/2 \rfloor$. `T_middle` copies that tensor; `T_dimer` contracts it with its right neighbor; a `combiner` fuses the two physical dimension-two indices into dimension four. These objects are local tensors in a particular MPS gauge, not reduced density matrices. Their entries depend on basis and virtual gauge choices even when the physical state is unchanged.

4.2 `OBC_Schwinger_dmrg.jl`: construction utilities

`extract_nonzero_elements` verifies that a matrix is square and returns a dictionary of nonzero entries. Its additional `nonzero_list` is unused. `assign_itensor_elements` finds link and physical indices by tags and writes a local operator into a selected virtual block. The rewritten builder passes explicit index identities instead, making physical input/output and left/right orientation unambiguous.

`schwinger_gauge` fills the three-state W_j operator described above. At site 1 it incorporates $\theta/(2\pi)$ into a_1. Intermediate sites carry q_j. The final tensor only propagates identities. `get_MPO_neutral`

allocates tensors and contracts the virtual boundary vectors selecting states 1 and 3. Its QN branch hard-codes dimension three rather than using the argument `D_ham`; the new implementation deliberately supports dense tensors only.

`create_custom_sites` is unused by the active driver. It creates parity-dependent QN indices. Its odd/even comments are reversed relative to the branch test, and its documented basis charges need auditing against ITensor's Up/Dn convention before use. It should not be enabled as an incidental GPU change. The commented-out fermionic MPO builder is also inactive.

`directsum_MPOs` manually block-adds the MPO tensors and reconnects virtual indices. This represents the sum without compressing it. The new code uses the library's `+(...;alg="directsum")` for the same operation, and small matrix comparisons check agreement with the original routine.

4.3 The original diagnostics and solver

`chrd_dist` computes $(2L/\pi)\sin(\pi j/L)$, a chord coordinate often used when examining open-chain entanglement profiles. Its use alone does not establish conformal scaling. `EEcalc` moves the canonical center, computes Schmidt singular values, and evaluates $-\sum p \log p$. Its loop starts at bond 2, leaving the first output equal to zero even for an entangled first bond. It also lacks a guard for $p = 0$. The new diagnostic covers every cut and skips zero probabilities.

`DemoObserver` remembers the previous energy and tests a relative change divided by $|E|$. That division is unstable at zero, and the same observer is reused across all target states in `Schwinger`. The new observer fixes both issues; this changes the stopping policy but not the Hamiltonian. The old `measure!` hook effectively does nothing.

`Schwinger` calculates μ, builds H, and runs ground-state DMRG. For state $k > 0$, it minimizes

$$H_{\text{pen}} = H + w \sum_{j<k} |\psi_j\rangle\langle\psi_j|, \qquad w = 100\sqrt{x}. \tag{13}$$

The penalty discourages overlap with previously found states. Its scale must exceed the relevant energy separation, and inaccurate previous states can contaminate later ones. The library's returned objective includes the penalty [3]. The new reports distinguish this objective from the physical energy $\langle H \rangle$ and save the overlap matrix. The original `Symm` allocation is unused, and `theta`, `spin`, and N are not used inside its solver to enforce any extra condition.

5 New files: what each one does

File	Purpose
`SchwingerGPU.jl`	Model, shared solver, device interface, diagnostics, configuration, reporting, and comparisons.
`CUDABackend.jl`	NVIDIA device selection, precision-preserving transfer, synchronization, and device metadata.
`run_cpu.jl`	Small command-line entry point selecting the host backend.
`run_cuda.jl`	Entry point selecting the NVIDIA backend.
`setup_cuda.jl`	Installs CUDA in the new project on the intended machine. Does not launch DMRG.

File	Purpose
`check_cuda.jl`	Real-device storage and precision checks plus an $N = 4$ CPU/GPU smoke comparison.
`benchmark.jl`	Explicit $N = 10, \ldots, 20$ comparison with warmup, repeats, and median timing summaries.
`tests/runtests.jl`	Tiny CPU tests against an independent full Hamiltonian and unchanged original helper routines.
`tests/check_syntax.jl`	Base-only parser check for every top-level Julia script, including CUDA scripts without loading CUDA.
`Project.toml`, `Manifest.toml`	Project dependencies and resolved package versions.
`README.md`	Concise commands, status, limitations, and option reference.
`docs/`	This PDF, editable source files, and `build_pdf.ps1` (two LaTeX passes to refresh links and contents).
`results/`	Local validation outputs; future run reports go here by default.

5.1 Configuration and the general backend interface

`Model` stores N, x, y, m, θ, g and the initial spin. `Solver` stores numerical settings separately, so changing accuracy controls cannot silently redefine the model. `validate` checks sizes and finite parameter values; `initial_labels` enforces spin parity and constructs the initial product state.

`Backend` stores a name, an upload function, a download function, a synchronization function, and metadata. This is the general version requested alongside NVIDIA: the algorithm depends on these capabilities rather than a GPU vendor name. The CPU adapter uses independent copies and a no-op synchronization. This avoids aliasing a mutable initial state between runs.

An illustrative interface use is:

```
backend = Backend("my_device", upload_tensor_network,
                  download_tensor_network, synchronize_device)
run_case(model, solver, backend)
```

This is an interface example, not executable vendor support. A new backend must provide compatible ITensor storage and all operations reached by the solver, retain Float64, and pass the same tests. Metal's precision restrictions and backend decomposition limitations mean that simply replacing a package name is insufficient [1].

5.2 Hamiltonian and starting state

`gauge_mpo` directly implements W_j with explicit link, output, and input indices. Its nested `block!` writes a 2×2 operator into a chosen virtual block. All such scalar writes occur on the CPU before any upload. `build_problem` creates dense spin sites, the hopping and mass OpSums, the gauge MPO, the direct-sum Hamiltonian, and the product MPS. It returns the individual pieces as well as the total for validation and reuse.

There are no model-specific GPU kernels to maintain. MPO preparation is small compared with large-bond DMRG, and the new code uploads the complete prepared MPO and MPS once per solve. The gauge MPO's small virtual dimension is retained. Production runs never call the independent dense-matrix test routine.

5.3 The NVIDIA adapter

`cuda_backend` checks `CUDA.functional()`, selects a zero-based device ID, and disables accidental scalar indexing on GPU arrays. It uploads using

```
adapt(CUDA.CuArray, tensor_network)
```

and downloads with `adapt(Array,tensor_network)`. Using `adapt` explicitly matters: the convenient `cu(...)` conversion normally changes floating-point input to Float32 [2]. A CPU/GPU comparison that changes precision at the same time would conflate two experiments.

Tensor index bookkeeping and Julia control flow remain on the CPU. The storage selected by the adapter dispatches the heavy numerical operations to the GPU libraries through ITensor's extensions. CUDA synchronization is called before and after timing to wait for asynchronous device work. The optional cuTENSOR package is not required or installed by this implementation; the basic CUDA path is the initial target [1].

The adapter records the CUDA package version, device name, device ID, and precision. It raises an error when CUDA is unavailable; it never silently labels a CPU fallback as GPU timing. Device-storage assertions in `check_cuda.jl` are intended to catch an unsuccessful conversion before the larger comparison.

5.4 Optimization and observables

`EnergyObserver` holds the convergence history and one maximum truncation error per sweep. Its `measure!` hook accumulates errors from individual bonds; its `checkdone!` hook implements the zero-safe stopping rule. These are separate from the diagnostics that evaluate the final state.

`solve` seeds the host random generator, synchronizes the device, and times the loop over target states. Every optimization starts from a fresh copy of the same input MPS. Ground-state DMRG uses H; later states use the orthogonality penalties. Physical expectations are computed after the DMRG timing interval. Host seeding does not guarantee bitwise identity across hardware: decomposition algorithms and reduction orders can differ, even for identical initial tensors.

`diagnostics` downloads each final state and the Hamiltonian once, then computes on the host:

$$n_\psi = \langle\psi|\psi\rangle, \qquad E = \frac{\langle\psi|H|\psi\rangle}{n_\psi}, \tag{14}$$

$$\sigma_H^2 = \frac{\langle\psi|H^2|\psi\rangle}{n_\psi} - E^2, \qquad r = \sqrt{\max(0, \sigma_H^2)}. \tag{15}$$

For a Hermitian H, r is the norm of the eigenstate residual divided by $\|\psi\|$ in exact arithmetic. The calculation evaluates $\langle H\psi|H\psi\rangle$ by a tensor-network contraction without explicitly constructing an H^2 MPO. The raw variance is saved because floating-point cancellation can make a tiny variance negative. A clipped zero is not a certificate of exactness; near the roundoff floor, compute a more stable residual or use higher precision if needed.

The same diagnostic records total S^z, actual maximum MPS bond dimension, completed sweep count, energy-stopping flag, and absolute normalized overlaps among target states. The flag indicates the specified energy test, not proof of ground-state convergence. A residual can be small for an excited eigenstate too, so it must be interpreted with the variational search and tiny exact checks.

`entropy_profile` uses host MPS tensors, visits every bond, squares Schmidt singular values, normalizes their probabilities, and skips zero entries in the logarithm. It returns $S_b = -\sum_a p_{b,a} \log p_{b,a}$

(natural logarithms, in nats) and the original chord coordinate. `middle_tensors` returns the canonical middle tensor, two-site tensor, fused tensor, and combiner, using a copy to avoid changing the caller's MPS gauge.

5.5 Running, parsing, and saving

`run_case` validates the configuration, builds or accepts a shared problem, times transfers, calls the solver, and calculates diagnostics. `write_result` writes TOML containing model and solver fields, software versions, backend metadata, times, per-state observables, and overlaps. The helper `fields` converts a configuration struct into a dictionary.

`parse_options` accepts only `--key=value` arguments, rejects duplicates and unknown option names, and `options_config` converts strings into typed parameters. Odd-chain defaults are resolved there. `main` selects BLAS thread count, obtains the requested backend, runs a case, and prints energies and convergence status. The two runner files only choose the backend; they do not duplicate the physics.

The result object in Julia contains MPS states. TOML files contain scalar data and histories, not wavefunction checkpoints. Single-run default filenames are reused; choose a distinct `--output` path to retain multiple runs. The benchmark creates a unique output directory by default. Recorded timestamps use the executing machine's clock; keep that machine's timezone with research records if comparing runs across sites.

5.6 Comparison runner

`compare_case` builds one host problem and uses its identical physical index identities and initial state for both backends. It saves the two run reports and a comparison report. In addition to energy differences, it computes the phase-independent normalized fidelity

$$F = \frac{|\langle\psi_{\mathrm{CPU}}|\psi_{\mathrm{GPU}}\rangle|^2}{\langle\psi_{\mathrm{CPU}}|\psi_{\mathrm{CPU}}\rangle\langle\psi_{\mathrm{GPU}}|\psi_{\mathrm{GPU}}\rangle}. \tag{16}$$

The initial energy-agreement rule is $|E_G - E_C| \leq 10^{-8} + 10^{-9}|E_C|$. This is a practical comparison threshold, not an uncertainty estimate. Near a degeneracy, two valid ground-state vectors can have a lower fidelity while spanning the same low-energy subspace; compare subspaces or suitable observables before calling that a numerical failure.

`benchmark` loops over every integer in the requested range. It performs one unmeasured full run at each size for each backend, followed by three measured repeats by default. It summarizes the median DMRG time on each backend and the ratio of medians. CPU currently runs before GPU within each pair; order is not randomized, so thermal or background-load effects should be checked in a later performance study.

6 Correctness checks and their limits

6.1 Independent tests before the study

The test file constructs a dense matrix for $N \leq 6$ using spin configurations and bit flips, independently of the MPO builder. Each diagonal is the sum of electric, staggered mass, and interaction energies. Each neighboring opposite-spin pair produces an off-diagonal hop. This tests the physical signs, $\frac{1}{2}$ factors, open boundary, xg coefficient, background angle, and constant energy offset.

For $N = 2, 3, 4, 5, 6$ and $\theta = 0, \pi, 0.73$, the tests use nonzero g, nonunit y, and a different mass, so agreement is not confined to the default zero-interaction case. The entire matrix is compared to the new MPO. The original gauge MPO and manual direct sum are independently compared to the new versions. The unchanged original helper is included inside a Julia module; its driver, which would launch $N = 20$, is never executed by this suite.

Small DMRG calculations for $N = 4, 5, 6$ compare ground-state energies with full diagonalization, and check normalization, residuals, stopping flags, and entropy at all cuts against an independent full-state SVD. The $N = 4$ case also calls the original `Schwinger` routine. A two-state $N = 4$ test checks physical excited energies and overlap. Product-state entropy and the dimension-four fusion are tested too. A host adapter exercises the general comparison/reporting interface without pretending to be a GPU.

These correctness tests use increased sweeps/local iterations, sufficient bond dimension, and zero noise, rather than claiming the original ten-sweep settings are converged. The original solver retains its internal settings when called. Exact tests are intentionally tiny: at $N = 20$ a dense real matrix would have 2^{40} entries, requiring about 8 TiB just for its entries. Matrix-free exact methods or restricted sectors are a different possible project, not part of the production DMRG port.

6.2 Observed local validation

The full local suite completed successfully: **115 checks passed**, with no failures or errors. The environment was Julia 1.12.4, ITensors 0.9.31, and ITensorMPS 0.4.1, using four BLAS threads and one Julia thread. All seven new top-level Julia scripts also passed a Base-only syntax check, including the NVIDIA files; parsing is not a CUDA execution test.

N	CPU DMRG ground energy	Exact ground energy	Absolute difference
4	-21.854627410289930	-21.854627410289932	3.55×10^{-15}
5	-26.717539977796100	-26.717539977796104	3.55×10^{-15}
6	-33.965949729130365	-33.965949729130350	1.42×10^{-14}

Values are printed to finite decimal precision; differences were computed from the underlying floating-point values. These near-roundoff differences establish tiny-system agreement, not an error estimate for larger chains. The checks comprised 8 configuration checks, 60 full-Hamiltonian/reference checks, 40 DMRG/observable checks, 5 excited-state/convergence checks, and 2 generic-adapter/report checks. All three ground-state energy stopping flags were true, and all tested residual norms were below 5×10^{-6}. Variance subtraction at such small errors is roundoff-limited.

The unchanged original CPU solver at $N = 4$ returned -21.854627410289932 after two sweeps, agreeing with the new result. The two lowest $N = 4$ energies and their mutual overlap also passed the independent excited-state assertions. The first excited energy was -15.31231201412561 with measured total $S^z = -1$, despite initialization at spin zero: this directly illustrates that the dense solver does not enforce the initial spin sector. Per-state results are saved in `results/validation_N4.toml`, `validation_N5.toml`, `validation_N6.toml`, and `validation_excited_N4.toml`. The complete test transcript is `results/validation_log.txt`. These first-run timings include compilation effects and are not presented as performance measurements.

Not performed: NVIDIA execution, a CPU/GPU numerical comparison, or the $N = 10, \ldots, 20$ study. Windows hardware inspection reported Intel UHD Graphics 620. The code and smoke-check script for an NVIDIA machine are supplied, but a real device is required to validate that path.

6.3 GPU checks that remain to be run

On a real NVIDIA device, `check_cuda.jl` must verify Float64 MPO storage on GPU and an $N = 4$ comparison. Extend this to nonzero g, nonzero background angles, odd chains, and the excited-state branch before relying on those modes for research. The current CUDA adapter and library dispatch have not been exercised on this Intel-only machine. A successful CPU test validates the shared mathematics and control flow, not vendor-specific execution.

7 Commands for review, small checks, and the later study

From the repository root, enter `GPUimplementation/` first. Run all the following commands inside that folder. The unchanged local reference sources used by the tests are bundled in `reference/`. The saved manifest began from this machine's installed Julia environment. The setup step resolves and installs the new project's CUDA dependency on the target machine; preserve its resulting manifest with future measurements.

7.1 Portable CPU execution

```
julia --project=. -e 'using Pkg; Pkg.instantiate()'
julia --project=. tests/runtests.jl
julia --project=. run_cpu.jl --N=4 --maxdim=16 --nsweeps=12
```

The first Julia execution may spend substantial time precompiling. That compilation must not be interpreted as DMRG runtime. In this editing session, restricted Windows execution prevented Julia from opening precompilation subprocess pipes, so the validation needed execution outside that sandbox. This is an execution-environment issue, not a GPU requirement.

7.2 NVIDIA setup and first real-device check

```
julia --project=. setup_cuda.jl
julia --project=. check_cuda.jl
julia --project=. run_cuda.jl --N=4 --maxdim=16 --nsweeps=12
```

Use a machine with an NVIDIA GPU, a suitable driver, and Julia. The CUDA package normally manages its runtime dependencies [4]. Device IDs are zero-based; select another device with `--device=1`. No CUDA installation can turn the detected Intel UHD Graphics 620 into an NVIDIA device.

7.3 After reading the guide: $N = 10, \ldots, 20$

```
julia --project=. benchmark.jl --sizes=10:20 --repeats=3 --warmup=true
```

This includes odd sizes. If the physical question instead requires zero initial spin at every size, compare the even subsequence separately. Do not silently label a half-integer odd-chain calculation as spin zero. The benchmark uses the original model and solver settings, with the documented observer corrections. It compares the new CPU and GPU paths; tiny tests provide the link back to the original implementation.

If energy, residual, or bond-dimension convergence is inadequate, repeat both backends with improved controls, for example:

```
julia --project=. benchmark.jl --sizes=10:20 --nsweeps=30 --maxdim=200 --eig_maxiter=5 --
    noise=0 --repeats=3
```

This is a separate convergence experiment. Turning noise off may change optimization robustness; do not accept it automatically as a better ground-state search. For strict fixed-sweep timing, add `--energy_tol=0` to both paths. To extend the size range later, change `--sizes=22:40`; first check convergence and memory at a few individual sizes.

7.4 Options and interpretation of outputs

Model options are `N,x,y,mass,theta,g,spin`. Solver options are `nsweeps,maxdim,cutoff,noise,energy_tol,min_sweeps,krylovdim,eig_maxiter,eig_tol,num_states,seed,outputlevel`. Runtime options include `blas_threads`, `device`, and `output`. Benchmark controls are `sizes,repeats,warmup,directory`. Theta must be a numeric value in radians; its default is π, but the parser does not evaluate expressions such as `--theta=pi`.

Read per-state `energy`, `variance_raw`, `residual_norm`, `energy_converged`, `maxlinkdim`, and `sweep_max_truncation_error` together. Comparison reports add energy agreement and fidelity. A large speedup is not useful if either solution is inaccurate. Conversely, a slightly different stopping sweep can change total runtime without indicating faster per-update kernels.

8 Performance expectations and a defensible extended study

8.1 Why small GPU runs can be slower

GPU parallelism helps when contractions and decompositions are large enough to amortize launch and scheduling overhead. At small N and modest χ, many matrices are small; a CPU can finish them before GPU overhead is recovered. The two-site DMRG sweep also has a sequential dependency along the chain. GPU acceleration applies within numerical operations, not by updating all bonds independently.

As a rough guide, an interior two-site tensor has $d^2\chi^2$ entries, and decompositions of matrices with dimensions proportional to $d\chi$ have cubic work in that scale. Actual costs also depend on MPO dimension, environment contractions, noise, eigensolver iterations, and library workspaces. The memory model includes MPS tensors, MPO tensors, left/right environments, Krylov vectors, and decomposition scratch space. Device memory capacity alone does not establish feasible N or converged χ.

8.2 What is timed

Reports distinguish host Hamiltonian construction, upload/copy time, and synchronized DMRG time. The DMRG timer excludes physical-energy recomputation, host downloads, full diagnostics, report writing, and process startup. It includes observer work and, for multiple states, all penalized optimizations. Total end-to-end latency is therefore greater. First-run times may include compilation; same-size warmup is enabled for the benchmark.

For an interpretable comparison, keep model parameters, precision, initial tensors, solver settings, and CPU BLAS threads identical. Record the machine and driver, the resulting manifest, warmup choice,

number of repeats, and background load. The script records much of the software/device configuration; `CUDA.versioninfo()` supplies additional runtime/driver context in the smoke-check output. Thread counts and a single median do not fully characterize noise, so retain individual repeats.

8.3 Suggested research progression

First establish tiny-system exact agreement, then real-device agreement, then the requested $N = 10$ through 20 baseline. Next study convergence in χ, sweep count, local eigensolver accuracy, and initialization. Only then examine larger chains and physical changes such as m, θ, or g. Useful observables include energy density, gaps with verified orthogonality, spatial charge/electric profiles, and entanglement scaling.

Separate numerical extrapolation from physical finite-size extrapolation. Changing the mass convention, adding constant terms, enforcing a different spin sector, or switching precision changes the comparison. The current output provides energies and basic diagnostics; richer observables and portable MPS checkpointing are natural follow-up implementations, not data already produced. At $\theta = \pi$ or near competing low-energy states, use multiple starts and subspace-sensitive diagnostics before making claims about gaps or phases.

9 Sources and reproducibility

The model derivation and original-code audit above are based on the supplied local Julia files. Library behavior was checked against official documentation and the installed package source. The following are reference URLs; GPU capabilities can change between package versions.

1. ITensor GPU backends: https://docs.itensor.org/ITensors/v0.7/RunningOnGPUs.html. A versioned capability reference; verify the packages installed on the target machine.
2. CUDA.jl memory and type-preserving upload: https://cuda.juliagpu.org/stable/usage/memory/.
3. ITensorMPS DMRG API, settings, and penalized energies: https://docs.itensor.org/ITensorMPS/stable/DMRG.html.
4. CUDA.jl installation: https://cuda.juliagpu.org/stable/installation/overview/.

The reference files' SHA-256 values are recorded in `original_files.sha256`. They allow a later check that the original sources remained unchanged. The new project owns its dependency files; the user's global Julia project is not edited by the provided setup script.